FA2 Instructions: Design to Deployment

FA Activity: Design to Deployment, OOMSD Presentation (FA2)

Marks: 10 | **Units covered:** Unit 3 (System and Service Design) and Unit 4 (Architectural Integration and Deployable System Design)

In FA2 you take the same system used in FA1.

No.	Assessment criterion	Marks
1	Architecture and scalability design	2
2	REST API design and UI approach	2
3	Design patterns and quality attributes	2
4	Deployment design and integration	2
5	ADRs and AI-assisted design review	1
6	Presentation, live demo and viva	1
	Total	**10**

Instructions for Students

Design sequence: FA1 System and Scenario → Architecture Pattern → REST API → Scalability Plan → UI Approach → Design Patterns → Quality Attributes → Deployment Diagram → Integration View → ADRs and AI Review

1. **Group and case study**
    - Work in the same group of 2 to 3 students and on the same system as FA1.
2. **Architecture pattern**
    - Choose a main architecture pattern: Layered, Client-Server, Event-Driven or Microservices (a hybrid is allowed).
    - Draw a high-level architecture diagram showing components and how they talk to each other.
    - Justify the choice for your scenario and state one pattern you rejected and why.
    - If you choose Microservices, show the service boundaries. Do not split into services without a reason.
3. **REST API design**
    - Model at least 4 resources from your Class Diagram (for example orders, users, products, payments).
    - Give a table of endpoints with HTTP method, URL, purpose and status codes. Use GET, POST, PUT or PATCH, and DELETE correctly.
    - State your API versioning approach (for example /api/v1/ or a version header).

- Show where an API Gateway sits and what it does for you (routing, authentication, rate limiting).
- Document at least 5 endpoints in an OpenAPI or Swagger file, or test them in Postman.

4. **Scalability and performance**
   - Explain where vertical scaling is enough and where horizontal scaling is needed.
   - Show where a load balancer sits and the balancing method you use (round robin, least connections).
   - Choose a caching strategy (cache-aside, write-through or TTL-based) and say exactly what you will cache.
   - Explain database scalability for your tables: indexing, read replicas, partitioning or sharding.
   - Say whether a CDN helps your system and what it would serve.
   - Link every choice to the numbers in your scenario.
5. **UI design approach**
   - Compare traditional HTML-based UI, React-based UI and HTMx for your system.
   - Choose one and justify it. Show at least 2 wireframes or screens (hand-drawn, Figma, draw.io, or a built page).
   - Show which API endpoints each screen calls.
6. **Design patterns**
   - Apply at least 3 patterns from Factory, Singleton, Adapter, Observer, Strategy and MVC to your system.
   - For each pattern: the problem in your system, where it is used, and a small class diagram or code snippet.
   - Examples: Adapter for a payment gateway, Observer for notifications, Strategy for pricing or discount rules, Factory for creating objects of different types.
   - Patterns must be consistent with your FA1 Class Diagram. Show any new classes or changes.
7. **Architecture quality attributes**
   - Evaluate your design on performance, scalability, maintainability and security.
   - For each attribute give one concrete tactic you used (for example caching for performance, stateless services for scalability, layering for maintainability, authentication and input validation for security).
   - Name one trade-off in your design (for example caching speeds reads but may serve stale data).
8. **Deployment design**
   - Draw a UML Deployment Diagram showing nodes, artifacts, the database, the load balancer and the API gateway.
   - Explain containerization for your system: which components go into which container, and what would be in a Dockerfile.
   - A working docker-compose file or running container is a bonus, not mandatory.
9. **Integration view**
   - Show how the layers connect from end to end: UI layer → API layer → object model → database → deployment.
   - Trace one use case from FA1 through all layers (for example Place Order: screen, endpoint, class methods, tables, container).
   - Make sure names are consistent across the Class Diagram, ER Diagram, API resources and UI screens.

10. **Architecture Decision Records (ADRs)**
    - Write at least 3 ADRs for your key decisions (for example architecture pattern, caching strategy, UI approach).
    - Use the template in section 3. You may draft with AI, but the context, alternatives and consequences must reflect your own reasoning.
11. **Presentation**
    - Each group presents the complete design in 8 to 10 minutes, followed by a short viva.
    - Every member must explain and justify any part of the design.
    - Cover: scenario, architecture, API, scalability, UI, patterns, quality attributes, deployment, integration trace and ADR highlights.

ADR template (one page per decision)

Field	What to write
Title	Short name, for example ADR-001 Use layered architecture with a REST API
Status	Proposed, accepted or superseded
Context	The problem, your scenario numbers and constraints
Options considered	At least 2 alternatives with one pro and one con each
Decision	What you chose, in one or two sentences
Consequences	Benefits, costs, risks and what you will need to watch

Assessment Rubric (10 Marks)

No.	Criterion	Excellent	Good	Fair	Poor
1	Architecture and scalability design (2)	2: pattern clearly justified against the scenario numbers, rejected option explained; scaling, load balancing, caching, database scaling and CDN all chosen and linked to the load	1.5: sound pattern and most scaling choices justified, one weak link to the scenario	1: pattern chosen but justification generic, scaling choices listed without reasons	0.5: unclear or unsuitable pattern, scalability missing
2	REST API design and UI approach (2)	2: 4 or more well-modelled resources, correct HTTP methods and status codes, versioning and gateway shown, API documented or tested; UI approach compared, justified,	1.5: mostly correct API and UI with minor errors	1: verbs used in URLs or wrong methods, UI choice not justified	0.5: API incomplete or incorrect, no UI reasoning

No.	Criterion	Excellent	Good	Fair	Poor
		screens linked to endpoints			
3	Design patterns and quality attributes (2)	2: 3 or more patterns correctly applied to real problems in the system with diagram or code, consistent with the Class Diagram; four quality attributes evaluated with tactics and a trade-off	1.5: patterns correct, quality attribute analysis partly specific	1: patterns named but forced or incorrect, quality attributes generic	0.5: patterns misused or missing, no real quality analysis
4	Deployment design and integration (2)	2: correct deployment diagram, containerization explained, end-to-end trace of one use case through UI, API, object model, database and deployment with consistent names	1.5: correct diagram, trace has small gaps or naming inconsistencies	1: diagram incomplete, layers described separately without a trace	0.5: deployment unclear, no integration
5	ADRs and AI-assisted design review (1)	1: 3 ADRs with context, alternatives and consequences in own reasoning; AI log shows critical use with accepted and rejected suggestions	0.75: 3 ADRs, alternatives thin, AI log present	0.5: fewer than 3 ADRs or copied AI text, log is only a list of prompts	0.25: ADRs or log missing
6	Presentation, live demo and viva (1)	1: within 8 to 10 minutes, demo from the actual tools, every member explains and justifies any part confidently	0.75: clear presentation, demo partly from screenshots, minor gaps in viva	0.5: reads slides, uneven participation, weak viva answers	0.25: unprepared or one member carries the group